

DD: BIL-02-TAX-01 — Tax Invoice & Gateway

Developer implementation guide: DD_BIL-02-TAX-01-Tax_Invoice_and_Gateway-DEV_IMPL_GUIDE-v1.4.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract
DD_BIL-02-TAX-01-Tax_Invoice_and_Gateway-v1.4.md.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v1.4 Satellite of **BIL-02 Invoicing (Core)**. Owns the generation, signing, and lifecycle of tax invoices — the legal documents that prove tax was declared on received customer payments, signed by the operator's tax authority gateway. Includes the pluggable gateway pattern, retry framework, and admin operations. **Sibling DDs:** BIL-02-GEN-01 Invoice Generation, BIL-02-ADJ-01 Invoice Adjustments, BIL-02-READ-01 Invoice Read API, BIL-02-STA-01 Status Management & Recovery. **Version history:** see CHANGELOG_BIL-02-TAX-01-Tax_Invoice_and_Gateway.md in the same directory.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-02-TAX-01-Tax_Invoice_and_Gateway-v1.4.md
Version	v1.4
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- GET /api/tax-invoices/dashboard?operator={code}
- GET /api/tax-invoices/{id}
- GET /api/tax-invoices/{id}/pdf
- GET /api/tax-invoices/{id}/signing-history
- POST /api/cycles/.../settle
- POST /api/payments/...

- POST /api/tax-invoices/manual
- POST /api/tax-invoices/{id}/cancel
- POST /api/tax-invoices/{id}/cancel/approve
- POST /api/tax-invoices/{id}/resolve-no-action
- POST /api/tax-invoices/{id}/retry-signing

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- billing-invoicing
- billing-invoicing-tax

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- InvoiceGenerator
- PaymentApplied
- PaymentReceived

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing & Revenue | **Version:** v1.4

Satellite of **BIL-02 Invoicing (Core)**. Owns the generation, signing, and lifecycle of tax invoices — the legal documents that prove tax was declared on received customer payments, signed by the operator's tax authority gateway. Includes the pluggable gateway pattern, retry framework, and admin operations.

Sibling DDs: BIL-02-GEN-01 Invoice Generation, BIL-02-ADJ-01 Invoice Adjustments, BIL-02-READ-01 Invoice Read API, BIL-02-STA-01 Status Management & Recovery.

Version history: see `CHANGELOG_BIL-02-TAX-01-Tax_Invoice_and_Gateway.md` in the same directory.

Glossary

Term	Meaning
Tax invoice	A legal document, distinct from the regular cycle invoice, that proves tax was declared on a received customer payment. Issued whenever the customer pays money to the operator — for POSTPAID customers this is when they pay a cycle/one-off invoice; for PREPAID customers this is when they top up their wallet (or pay each cycle directly if the operator doesn't use wallets). The tax invoice carries the tax authority's signed reference number and (in most jurisdictions) a QR code that lets the customer verify the invoice with the authority. Required by tax regulators in operators that mandate it.
Payment moment	The instant money moves from customer to operator. This is when tax is collected (the customer's payment is tax-inclusive — a portion of every payment is tax owed to the authority). The tax invoice declares that tax. Payment moments include: POSTPAID customer paying an invoice (full or partial), PREPAID customer topping up their wallet, PREPAID customer making a direct per-cycle payment (no-wallet operator model).
Tax authority	The regulator in each country that enforces tax invoicing requirements, where one exists. Some countries require electronic tax invoice signing through a national platform; others do not. TAX-01 supports both deployment types: where there is an authority, the per-operator signer integrates with it; where there is none, tax invoice generation is simply disabled for that operator.
Tax Gateway	The TAX-01 component that talks to the tax authority's API where one exists. Pluggable per operator — each operator has its own signer implementation that knows the authority's protocol, payload format, certificate handling, etc.
Signed invoice number	A reference number issued by the tax authority when it successfully signs a tax invoice. The customer sees this on the tax invoice PDF and uses it to verify with the authority. Distinct from the operator-internal <code>legal_invoice_number</code> (which is allocated before signing). Format varies per authority.
Tax signature payload	The raw data returned by the tax authority's signing response — typically containing the signed reference number, QR code data, timestamp, signer certificate info, optional cryptographic signature. Stored on the tax invoice for audit and PDF rendering.

Term	Meaning
QR signature data	A QR-code-encodable string returned by the tax authority allowing a third party (customer, auditor) to verify the tax invoice independently by scanning. Format varies by authority.
Pluggable signer	A Java implementation of the <code>TaxInvoiceSigner</code> interface, one per operator that has tax invoicing enabled. Selected at runtime based on <code>operator_code + invoice_type_operator_config.signing_service_implementation_ref</code> .
Signing failure	The tax authority rejected the submission (malformed payload, missing field, authentication failed, customer's tax ID not registered, etc.) or the gateway is unreachable. Triggers retry per the failure framework.

A — Target

1. What this process is about

In most jurisdictions where these operators run, the tax authority requires that for every payment received from a customer, the operator must produce a separate legal document called a **tax invoice** that proves tax (VAT, excise, etc.) was declared on the payment. The customer's payment is tax-inclusive: a portion of every payment they make is tax that the operator collects and remits to the authority. The tax invoice is how the operator declares that collected tax.

This is distinct from the regular cycle invoice — the cycle invoice tells the customer "you owe X for cycle Y"; the tax invoice tells the authority "we received payment of Z, of which T is tax."

The tax invoice carries a **signed reference number** issued by the tax authority's electronic invoicing platform where one exists. To get this reference, the operator submits the tax invoice payload to the authority's API; the authority validates, signs, and returns a unique reference plus QR code data. The customer's tax invoice PDF displays the signed reference and QR code so anyone can verify it with the authority.

TAX-01 is built to support both deployment types: operator countries that require electronic tax invoice signing, and operator countries that do not. Where signing is required, the per-operator signer integrates with the local authority's platform. Where it is not, the operator simply disables tax invoice generation.

Tax invoice is triggered by every customer payment. Tax is collected at the moment money moves from customer to operator, regardless of what that payment is for (cycle invoice, wallet topup, direct per-cycle pay) and regardless of billing mode. Three payment moments fire three different events that TAX-01 consumes:

Customer model	Payment moment	Event TAX-01 consumes	Tax invoice covers
POSTPAID	Customer pays a cycle or one-off invoice (full or partial) via M-Pesa, bank, card, cash	<code>PaymentApplied</code> from BIL-01 Charging Engine	The paid amount (proportional to the parent invoice if partial)
PREPAID — wallet model	Customer tops up their wallet via M-Pesa, bank, card, cash. All PREPAID services (Internet, TV, Voice, etc.) flow through the wallet.	<code>WalletToppedUp</code> from BIL-05 Wallet and Topup	The full topup amount
PREPAID — direct-pay model (for operators that don't use wallets)	Customer pays for an upcoming cycle directly	<code>PaymentReceived</code> from BIL-01 Charging Engine	The full direct-pay amount

In all three cases the customer's payment is **tax-inclusive** — they paid X and X already contains the tax. The tax invoice decomposes X into base + tax components per PLM-CFG-02 Tax Configuration's cascade rules for the relevant service.

TAX-01 owns this end-to-end: receive the payment event, generate the tax invoice with proper tax decomposition, submit to the right gateway, persist the signed response, retry transient failures, escalate persistent failures, and emit events so downstream consumers (notification, BIL-01 Charging Engine, analytics) can react.

What TAX-01 explicitly does not own

- **Regular cycle invoices and one-off invoices** — BIL-02-GEN-01 Invoice Generation
- **Adjustments (credit/debit notes)** — BIL-02-ADJ-01 Invoice Adjustments
- **Tax computation rules** (which taxes apply, rates, exemptions, cascading) — PLM-CFG-02 Tax Configuration. TAX-01 stores the breakdowns and submits them to the authority; it does not compute tax.
- **Payment recording** — BIL-01 Charging Engine. TAX-01 only consumes payment events, doesn't record them.
- **Wallet mechanics** — BIL-05 Wallet and Topup. TAX-01 only consumes `WalletToppedUp` events; it doesn't manage the wallet itself.

- **Invoice PDF retrieval** — BIL-02-READ-01 Invoice Read API. TAX-01 emits TaxInvoiceIssued with full payload; PDF rendering and channel dispatch are owned by DD_NOT-01 Notification Service; PDF stream retrieval is owned by READ-01.

2. Business rules and tax invoice flow

Rules grouped by phase: T (trigger), G (tax invoice generation), S (signing flow), F (failure and retry), C (cancellation and re-signing), D (admin operations).

Rule group T — Trigger

ID	Rule	Triggered on
R-TAX-01-T-1	TAX-01 consumes <code>sophix.billing.payment.applied</code> from BIL-01 Charging Engine for POSTPAID payments. Payload: <code>subscription_id</code> , <code>invoice_id</code> (the cycle or one-off invoice paid against), <code>payment_id</code> , <code>paid_amount</code> , <code>payment_method</code> (M-Pesa/bank/card/cash), <code>payment_timestamp</code> , <code>operator_code</code> , <code>currency</code> .	POSTPAID payment trigger
R-TAX-01-T-2	TAX-01 consumes <code>sophix.billing.wallet.toppedup</code> from BIL-05 Wallet and Topup for PREPAID wallet topups. Payload: <code>subscription_id</code> , <code>wallet_id</code> , <code>wallet_type_code</code> , <code>service_category_code</code> (resolved from PLM-CFG-03 Wallet Configuration), <code>topup_amount</code> , <code>topup_method</code> (M-Pesa/bank/card/cash), <code>payment_ref</code> , <code>topup_timestamp</code> , <code>operator_code</code> , <code>currency</code> .	PREPAID wallet topup trigger
R-TAX-01-T-3	TAX-01 consumes <code>sophix.billing.payment.received</code> from BIL-01 Charging Engine for PREPAID direct-pay payments (operators that don't use wallets). Payload: <code>subscription_id</code> , <code>payment_id</code> , <code>target_cycle_start</code> , <code>target_cycle_end</code> , <code>paid_amount</code> , <code>payment_method</code> , <code>payment_timestamp</code> , <code>operator_code</code> , <code>currency</code> .	PREPAID direct-pay trigger
R-TAX-01-T-4	TAX-01 evaluates per-operator config to decide whether to generate a tax invoice: <code>invoice_type_operator_config.enabled</code> for the <code>TAX_INVOICE</code> type must be true for that operator. If disabled, TAX-01 acknowledges the event but generates nothing.	Per-operator enablement
R-TAX-01-T-5	The triggering event ID is captured as <code>triggering_event_ref</code> on the tax invoice. Idempotency: re-consumption of the same event (Kafka retry) returns the existing tax invoice if found instead of generating a duplicate.	Idempotency
R-TAX-01-T-6	If the operator has tax invoice enabled but the customer's <code>tax_identifier</code> is missing from the customer snapshot (the customer doesn't have a registered tax ID — common for individuals in some jurisdictions), TAX-01 may still generate the tax invoice with a default placeholder per the operator's signing implementation. The signer implementation handles jurisdiction-specific placeholder rules.	Missing tax ID handling
R-TAX-01-T-7	Admin can also trigger tax invoice generation manually via REST (e.g., TAX-01 was down during the original event). Endpoint: <code>POST /api/tax-invoices/manual</code> with <code>{ payment_id }</code> or <code>{ topup_event_id }</code> . The system loads the original payment context, builds the generation context, and runs the standard flow.	Manual generation

Rule group G — Tax invoice generation

ID	Rule	Triggered on
R-TAX-01-G-1	TAX-01 implements TaxInvoiceGenerator, a generator class that follows the BIL-02 Core InvoiceGenerator framework. Same canonical structure as GEN-01's generators: idempotency check, customer snapshot capture, line item building, sequence allocation, atomic write. Lives in this DD because of its tight coupling with the signing flow.	Generator framework conformance
R-TAX-01-G-2	Every customer payment is tax-inclusive — the amount the customer paid already contains the tax owed to the authority. The tax invoice decomposes that paid amount into base + tax components per PLM-CFG-02 Tax Configuration's cascade rules. The customer never owes anything beyond what they already paid; the tax invoice just declares the tax portion.	Tax-inclusive principle
R-TAX-01-G-3	For POSTPAID PaymentApplied (full or partial payment): the tax invoice covers the paid amount. The tax breakdown is proportional to the original invoice's tax breakdown — if the customer paid 54.66% of the invoice, the tax invoice shows 54.66% of each tax component. NOT a recomputation via PLM-CFG-02 — the tax was determined at charge time when the original invoice was generated; the tax invoice just confirms that the paid portion includes a proportional share of that tax. The tax invoice's line items mirror the original invoice's line structure (same service categories, packages, wallet types) but with proportional amounts. <code>original_invoice_id</code> points to the parent invoice. Every payment (partial or full) triggers its own tax invoice — multiple partial payments against the same parent invoice produce multiple tax invoices, each covering its specific paid portion.	POSTPAID payment (incl. partial)
R-TAX-01-G-4	For PREPAID walletToppedUp: the tax invoice covers the full topup amount. One line item with: <code>service_category_code</code> = wallet's service category (e.g., INTERNET, TV, VOICE), <code>wallet_type_code</code> = topup'd wallet's type, <code>package_ref</code> = synthetic value like <code>TOPUP_{WALLET_TYPE}</code> , <code>total</code> = topup amount (tax-inclusive). PLM-CFG-02 Tax Configuration's <code>TaxService.computeTax(total, customer_location, service_category)</code> is called with the inclusive total to decompose it into base + tax components per the cascade. <code>original_invoice_id</code> is NULL — there is no upstream invoice. <code>metadata.topup_event_id</code> and <code>topup_payment_ref</code> populated.	PREPAID wallet topup
R-TAX-01-G-5	For PREPAID PaymentReceived (direct-pay): similar to wallet topup but the payment is direct (no wallet). One line item with: <code>service_category_code</code> = the subscription's service category (resolved via PLM-CFG-01 Package Catalog), <code>package_ref</code> = the subscription's package, <code>total</code> = <code>paid_amount</code> (tax-inclusive), decomposed via PLM-CFG-02. <code>original_invoice_id</code> may be NULL (if there's no pro forma) or set to the corresponding pro forma's ID (for traceability). <code>metadata.payment_received_event_id</code> populated.	PREPAID direct-pay
R-TAX-01-G-6	A fresh customer snapshot is captured at tax invoice generation time per Core R-C-3. Different payments → different snapshots (the customer's <code>tax_identifier</code> or address may have changed between payments).	Fresh snapshot
R-TAX-01-G-7	The tax invoice's <code>billing_mode</code> is set from the trigger context: for <code>PaymentApplied</code> , copied from the original invoice (POSTPAID); for <code>walletToppedUp</code> and <code>PaymentReceived</code> , always PREPAID.	Billing mode capture

ID	Rule	Triggered on
R-TAX-01-G-8	The tax invoice's <code>legal_invoice_number</code> is allocated from the <code>TAX_INVOICE</code> sequence (gap-free per operator + fiscal year, same as other legal documents). Format is per-operator and configured in <code>invoice_type_operator_config.legal_number_format</code> . Allocated atomically with the write.	Legal numbering
R-TAX-01-G-9	After successful write, the tax invoice's status is <code>GENERATED</code> and <code>payment_status</code> is <code>PAID</code> (by definition — the payment that triggered it already happened). No due date. The signing scanner picks it up next.	Initial state
R-TAX-01-G-10	For <code>POSTPAID</code> with hybrid grouping: if the parent invoice belonged to a specific group of the operator's grouping policy, the tax invoice inherits that <code>grouping_config_ref</code> and <code>grouping_key_values</code> . The tax invoice covers only the paid portion of that one parent invoice. If a customer paid two sibling invoices as separate payments, two separate <code>PaymentApplied</code> events fire, producing two separate tax invoices.	Grouping policy inheritance (<code>POSTPAID</code>)
R-TAX-01-G-11	For <code>PREPAID</code> topup and direct-pay: tax invoice grouping is <code>SINGLE</code> dimension by default (one tax invoice per payment event, no group split). Operators can configure otherwise if their <code>invoice_grouping_config</code> for <code>TAX_INVOICE</code> specifies a different dimension.	Grouping policy (<code>PREPAID</code>)
R-TAX-01-G-12	Each line item on the tax invoice carries its full identifying metadata: <code>service_category_code</code> , <code>package_ref</code> , <code>wallet_type_code</code> . This enables downstream tax reporting that segregates collected tax per service type and per package — required for operator finance teams reconciling with tax authority filings.	Line-level segregation for reporting

Rule group S — Signing

ID	Rule	Triggered on
R-TAX-01-S-1	After tax invoice generation, <code>TAX-01</code> must submit it to the tax authority for signing. This is done via a pluggable signer pattern: the <code>TaxInvoiceSigner</code> interface has one implementation per operator. The implementation is registered as a <code>@Component</code> and selected at runtime based on <code>invoice_type_operator_config.signing_service_implementation_ref</code> for the operator + <code>TAX_INVOICE</code> type.	Pluggable signer
R-TAX-01-S-2	The signing flow is asynchronous from the generation flow. After generation, the tax invoice is in status <code>GENERATED</code> . A scheduled scanner (every 1 minute per operator) picks up tax invoices in <code>GENERATED</code> status and submits them to the appropriate gateway. The tax invoice status transitions to <code>PENDING_SIGNATURE</code> during submission.	Asynchronous signing
R-TAX-01-S-3	On successful signing, the gateway returns a payload containing at minimum: signed reference number, QR signature data, timestamp. The signer parses this into a normalized structure stored on the tax invoice's metadata JSONB as <code>tax_signature_data</code> . The tax invoice status transitions to <code>SIGNED</code> , the <code>signed_invoice_number</code> is set, and a <code>TaxInvoiceSigned</code> Kafka event fires.	Successful signing
R-TAX-01-S-4	On signing failure, the tax invoice transitions to <code>SIGNING_FAILED</code> and a row is recorded in <code>tax_invoice_signing_failure</code> with the failure detail. The failure type determines next steps (see rule group F).	Failed signing

ID	Rule	Triggered on
R-TAX-01-S-5	Each signer implementation owns its gateway's protocol details: REST vs SOAP, certificate handling, authentication (mTLS, OAuth, signed JWTs, etc.), payload schema mapping, response parsing, error code interpretation. The shared TAX-01 framework knows nothing about specific gateways — it just calls <code>signer.sign(taxInvoice)</code> and handles the abstracted response.	Gateway abstraction
R-TAX-01-S-6	Signer implementations must be deterministic in their submission ordering: the same tax invoice submitted twice must produce the same signed reference (some gateways are idempotent natively; for those that aren't, the signer uses a deduplication token derived from the tax invoice ID). This ensures Kafka retries don't produce duplicate signed references.	Idempotency at gateway level
R-TAX-01-S-7	Signing has a timeout (default 30 seconds per submission, operator-configurable). If the gateway doesn't respond in time, the signer aborts and the tax invoice falls into <code>SIGNING_FAILED</code> with <code>failure_type = GATEWAY_TIMEOUT</code> . The retry framework re-tries — gateways are often slow, not broken.	Timeout

Rule group F — Failure and retry

ID	Rule	Triggered on
R-TAX-01-F-1	Signing failures fall into categories: (a) transient — gateway unreachable, timeout, 5xx response, certificate temporarily invalid → auto-retry; (b) validation — gateway rejected the payload (missing field, wrong format, customer PIN not registered) → admin review required; (c) persistent — same validation error after 8 retries → escalate.	Failure categorization
R-TAX-01-F-2	A scheduled retry scanner runs every 5 minutes per operator scanning <code>tax_invoice</code> rows in <code>SIGNING_FAILED</code> status with <code>failure_type</code> in the transient category and <code>next_retry_at <= now()</code> . For each, it re-submits to the gateway. Success → status <code>SIGNED</code> . Continued failure → <code>retry_count</code> increments, <code>next_retry_at</code> extends via exponential backoff (5min, 15min, 30min, 1h, 2h, 4h, 8h, 24h).	Transient retry
R-TAX-01-F-3	For validation failures (category b), <code>retry_count</code> is NOT incremented automatically — the failure won't fix itself by re-submitting. The tax invoice stays in <code>SIGNING_FAILED</code> with <code>failure_type = VALIDATION_FAILURE</code> until admin reviews and acts. Admin actions: (i) fix the underlying issue (e.g., update customer PIN in CRM) and trigger manual re-sign; (ii) mark as <code>RESOLVED_NO_ACTION</code> (tax authority accepted offline reconciliation); (iii) escalate.	Validation failure handling
R-TAX-01-F-4	After 8 transient retries without success (~3 days), the tax invoice transitions from <code>SIGNING_FAILED</code> to <code>GAVE_UP_AUTO</code> . An alert fires for operator review. Admin can: (i) trigger another retry manually (<code>retry_count</code> reset); (ii) mark resolved with notes; (iii) escalate.	Give up after max retries
R-TAX-01-F-5	Failure detail is stored in <code>tax_invoice_signing_failure</code> (one row per signing attempt that failed), with: <code>attempt_number</code> , <code>failed_at</code> , <code>failure_type</code> , <code>gateway_response_code</code> , <code>gateway_response_body</code> , <code>error_message</code> , <code>retry_scheduled_at</code> . This audit trail is essential for diagnosing recurring issues with a specific gateway.	Failure audit
R-TAX-01-F-6	Operator alerting fires on: any individual tax invoice reaching <code>GAVE_UP_AUTO</code> ; signing failure rate exceeding 1% per operator-hour (configurable threshold); gateway responding with auth failures (suggesting expired certificate).	Alerts

Rule group C — Cancellation and re-signing

ID	Rule	Triggered on
R-TAX-01-C-1	A signed tax invoice can be cancelled only via an explicit admin action with elevated privileges (Tax Compliance Officer role + dual approval). Cancellation requires reconciliation with the tax authority — most authorities require notification of any cancelled tax invoice. The cancellation flow: admin submits cancellation request → operator's compliance officer approves → TAX-01 calls the gateway's cancellation API → on success, tax invoice transitions to CANCELLED with <code>cancel_reason_code</code> and <code>cancellation_reference</code> from the authority.	Signed cancellation
R-TAX-01-C-2	An UNSIGNED tax invoice (status GENERATED, PENDING_SIGNATURE, or SIGNING_FAILED) can be cancelled with regular admin privileges (no Tax Compliance Officer approval needed, no gateway call) since the authority has no record of it. Status transitions to CANCELLED, gateway is not notified.	Unsigned cancellation
R-TAX-01-C-3	Re-signing of a SIGNING_FAILED or GAVE_UP_AUTO tax invoice is supported via admin action — admin clicks "Retry signing"; system re-submits to gateway. The same tax invoice row is updated with the new signed reference (if successful). The <code>tax_invoice_signing_failure</code> audit trail preserves all prior failure history.	Re-signing
R-TAX-01-C-4	Bulk reversal of tax invoices is prohibited as a normal operation — too risky for tax compliance. The bulk reversal capability in BIL-02-GEN-01 Invoice Generation explicitly excludes TAX_INVOICE from its scope. To reverse multiple tax invoices, admins must process them one at a time via the signed-cancellation flow with per-invoice reconciliation.	Bulk reversal exclusion

Rule group D — Admin operations

ID	Rule	Triggered on
R-TAX-01-D-1	Admin console provides a Tax Invoice Operations view per operator showing: pending generation count, pending signature count, signing failure count by <code>failure_type</code> , gave-up count, average signing latency, gateway error rate. Used by ops team for daily monitoring.	Operations dashboard
R-TAX-01-D-2	Admin can drill into any tax invoice and see: full lifecycle history (state transitions with timestamps), all signing attempts (one row per attempt with failure detail), the signed payload from the gateway (raw and parsed).	Per-invoice detail
R-TAX-01-D-3	Admin can manually trigger generation for a payment that should have produced a tax invoice but didn't (e.g., TAX-01 was down when the payment was applied). Endpoint: <code>POST /api/tax-invoices/manual</code> with <code>{ payment_id }</code> . The system loads the payment, builds the generation context, and runs the standard flow.	Manual generation
R-TAX-01-D-4	Admin can mark a tax invoice as RESOLVED_NO_ACTION when offline reconciliation with the authority is the chosen path. Requires resolution notes. The tax invoice status remains SIGNING_FAILED or GAVE_UP_AUTO, but an annotation is added indicating offline resolution; no further automatic retries.	Offline resolution
R-TAX-01-D-5	All admin actions on tax invoices (manual retry, mark resolved, cancel signed, re-sign) are captured in <code>invoice_history</code> with the admin user ID, timestamp, and reason.	Audit trail

3. Module owner and impacted modules

Role	Module	What it does
Owner	New billing-invoicing-tax package within the billing-invoicing module on cluster C	Co-located with BIL-02 Core. Owns the <code>TaxInvoiceGenerator</code> , the pluggable <code>TaxInvoiceSigner</code> interface and per-operator implementations, the signing scanner, the retry scanner, and admin REST endpoints.
Upstream events consumed	BIL-01 Charging Engine (<code>PaymentApplied</code> , <code>PaymentReceived</code>), BIL-05 Wallet and Topup (<code>WalletToppedUp</code>)	Trigger sources for tax invoice generation: payment-applied for POSTPAID invoice payments, wallet-topped-up for PREPAID wallet topups, payment-received for PREPAID direct-pay.
External calls (synchronous)	Tax authority gateway APIs, one per operator that has tax invoicing enabled	Calls vary per implementation: REST, SOAP, signed JWTs over HTTPS, mTLS. Each signer owns its own protocol.
Internal callees (synchronous)	PLM-CFG-02 Tax Configuration (compute tax decomposition for PREPAID topup/direct-pay; not called for POSTPAID since tax is proportionally scaled from the parent invoice's already-computed tax breakdown), PLM-CFG-01 Package Catalog (resolve package → service category for direct-pay), PLM-CFG-03 Wallet Configuration (resolve wallet → service category for topup events), SUB-LM-01 Subscription Lifecycle Management (read subscription state), CRM/customer service (customer snapshot via Core), BIL-02 Core (sequence service, invoice writer, event publisher), BIL-01 Charging Engine (read parent invoice details for POSTPAID), BIL-02-GEN-01 Invoice Generation (read parent invoice for POSTPAID)	All synchronous calls.
Downstream events emitted	<code>TaxInvoiceIssued</code> , <code>TaxInvoiceSigned</code> , <code>TaxInvoiceSigningFailed</code> , <code>TaxInvoiceSigningGaveUp</code> , <code>TaxInvoiceCancelled</code>	Consumed by DD_NOT-01 Notification Service (delivery), BIL-01 (audit), analytics, ops alerting.
Frontend	Admin Console — Tax Invoice Operations dashboard, per-invoice detail, manual retry/cancel/regenerate	Built on BIL-02 Core's admin shell.
Engines	No Drools in TAX-01. Tax rules in PLM-CFG-02 Tax Configuration (called via REST).	—

4. Foundation references

DB → FOUNDATION_DB.md · Kafka → FOUNDATION_KAFKA.md · Auth → FOUNDATION_AUTH.md (introduces TAX_COMPLIANCE_OFFICER role). Framework: BIL-02 Invoicing (Core) + SUB-WF-FRAMEWORK-01 Subscription Workflow Framework.

Technical design

Module placement

Co-located in billing-invoicing module on cluster C. New sub-package tax. Shares BIL-02 Core's invoice table for tax invoice rows (a tax invoice IS an invoice with `invoice_type_code = TAX_INVOICE`, plus operator-specific signing metadata). Adds 1 new dedicated table (`tax_invoice_signing_failure`) and uses Core's existing tables for everything else.

Code structure

```

modules/billing-invoicing/
  src/main/java/com/sophix/billing/invoicing/tax/
    service/
      TaxInvoiceGenerator.java      ← implements InvoiceGenerator (Core)
      TaxInvoiceSigner.java         ← interface for pluggable signers
      TaxInvoiceSigningService.java ← orchestrates signer invocation
    signer/
      (one TaxInvoiceSigner implementation per operator that has tax invoicing enabled)
    scanner/
      SigningScanner.java           ← every 1 minute: picks GENERATED → submits to gateway
      RetryScanner.java             ← every 5 minutes: re-submits SIGNING_FAILED transient
    kafka/
      PaymentAppliedConsumer.java   ← consumes BIL-01 payment-applied events (POSTPAID payments)
      WalletToppedUpConsumer.java   ← consumes BIL-05 topup events (PREPAID wallet topups)
      PaymentReceivedConsumer.java  ← consumes BIL-01 payment-received events (PREPAID direct-pay)

```

```

rest/
  TaxInvoiceAdminResource.java      ← admin operations endpoints
domain/
  TaxSignaturePayload.java          ← normalized signing response structure
  TaxInvoiceSigningFailure.java
repository/
  TaxInvoiceSigningFailureRepository.java
src/main/resources/
  config/liquibase/changelog/billing-invoicing/
    050_tax_invoice_signing_failure.xml

```

Tax invoice end-to-end flow

The canonical happy path from customer payment to signed tax invoice. Three flows depending on payment source: POSTPAID payment-applied, PREPAID wallet topup, PREPAID direct-pay. After generation, the signing flow is the same for all three.

Flow A — POSTPAID payment applied (full or partial)

Step 1 – Payment event arrives

Customer pays a cycle/one-off invoice via M-Pesa, bank, card, cash, etc.

BIL-01 Charging Engine records the payment, applies it to the invoice, and emits:

sophix.billing.payment.applied with:

```
{ subscription_id, invoice_id, payment_id, paid_amount, payment_method,
  payment_timestamp, operator_code, currency }
```

Step 2 – TAX-01 consumes the event (R-TAX-01-T-1)

PaymentAppliedConsumer:

- Validates event payload
- Looks up invoice_type_operator_config for (operator_code, TAX_INVOICE); if disabled → ack and stop
- Loads the parent invoice row to get its line items, totals, grouping context
- Builds InvoiceGenerationContext and calls InvoiceGenerationService.generate(ctx)

Step 3 – TaxInvoiceGenerator runs (R-TAX-01-G-*)

- Idempotency check on triggering_event_ref (the PaymentApplied event ID)
 - Capture fresh customer_snapshot via CustomerSnapshotService
 - Compute payment ratio = paid_amount / parent_invoice.total_amount
 - For each parent invoice line item, build a corresponding tax invoice line:
 - same service_category_code, package_ref, wallet_type_code, description
 - subtotal = parent.subtotal × payment_ratio
 - tax_breakdown = scale each component proportionally (parent.tax_breakdown × payment_ratio)
 - total = parent.total × payment_ratio
 - Aggregate per-line tax breakdowns into invoice-level tax_summary
 - Build invoice header:
 - billing_mode = POSTPAID (copied from parent invoice)
 - customer_snapshot = fresh
 - triggering_event_ref = PaymentApplied event ID
 - original_invoice_id = parent invoice ID
 - grouping_config_ref = parent's grouping_config_ref (preserved)
 - grouping_dimension, grouping_key_values = copied from parent
 - due_date = NULL
 - status = GENERATED, payment_status = PAID
 - Allocate TAX_INVOICE sequence number atomically
 - Write invoice + line items + outbox event (TaxInvoiceIssued with full payload)
- Result: tax invoice row in status GENERATED, ready for signing.

Note: a subsequent partial payment against the same parent invoice produces a SECOND tax invoice with its own triggering_event_ref (different payment_id). Customer ends up with two tax invoices for the parent invoice, each declaring the tax on its own paid portion.

Step 4-6 – Signing, retries, downstream – see "Signing flow (all three flows)" below.

Flow B — PREPAID wallet topup

Step 1 – Topup event arrives

Customer tops up their wallet via M-Pesa / bank / card / cash. BIL-05 Wallet and Topup processes the payment, credits the wallet, and emits:

sophix.billing.wallet.toppedup with:

```
{ subscription_id, wallet_id, wallet_type_code, service_category_code (resolved from PLM-CFG-03),
  topup_amount, topup_method, payment_ref, topup_timestamp, operator_code, currency }
```

Step 2 – TAX-01 consumes the event (R-TAX-01-T-2)

WalletToppedUpConsumer:

- Validates event payload
- Looks up invoice_type_operator_config for (operator_code, TAX_INVOICE); if disabled → ack and stop
- Builds InvoiceGenerationContext (no parent invoice lookup needed – topup is self-contained)
- Calls InvoiceGenerationService.generate(ctx)

Step 3 – TaxInvoiceGenerator runs (R-TAX-01-G-*)

- Idempotency check on triggering_event_ref (the WalletToppedUp event ID)
- Capture fresh customer_snapshot
- Build ONE line item for the topup:

```

description = "Wallet top-up - {service_category_label}" (localized)
package_ref = synthetic value like "TOPUP_{WALLET_TYPE}"
service_category_code = wallet's service category (e.g., INTERNET, TV, VOICE)
wallet_type_code = topup'd wallet's type
total = topup_amount (tax-inclusive)
Call PLM-CFG-02 Tax Configuration TaxService.computeTax(total, customer_location, service_category)
    → returns tax_breakdown decomposing the inclusive total into base + tax components
    per the operator's tax rules for that service category
subtotal = the decomposed pre-tax base
tax_total = total - subtotal
- Build invoice header:
    billing_mode = PREPAID
    customer_snapshot = fresh
    triggering_event_ref = WalletToppedUp event ID
    original_invoice_id = NULL
    metadata.topup_event_id, topup_payment_ref, topup_method
    grouping_config_ref = SINGLE-dimension group for TAX_INVOICE (or per operator config)
    due_date = NULL, status = GENERATED, payment_status = PAID
- Allocate TAX_INVOICE sequence number atomically
- Write invoice + line items + outbox event
Result: tax invoice row in status GENERATED.

```

Step 4-6 – Signing, retries, downstream – see "Signing flow (all three flows)" below.

Flow C — PREPAID direct-pay (no wallet)

Step 1 – Payment event arrives

PREPAID customer (operator doesn't use wallets) pays for an upcoming cycle directly via M-Pesa / bank / card / cash. BIL-01 Charging Engine records the payment and emits:
sophix.billing.payment.received with:
{ subscription_id, payment_id, target_cycle_start, target_cycle_end, paid_amount, payment_method, payment_timestamp, operator_code, currency }

Step 2 – TAX-01 consumes the event (R-TAX-01-T-3)

PaymentReceivedConsumer:
- Validates event payload
- Looks up invoice_type_operator_config for (operator_code, TAX_INVOICE); if disabled → ack and stop
- Resolves the subscription's package and service category via PLM-CFG-01 Package Catalog
- Optionally looks up the corresponding pro forma (if one exists) for traceability
- Builds InvoiceGenerationContext and calls InvoiceGenerationService.generate(ctx)

Step 3 – TaxInvoiceGenerator runs (R-TAX-01-G-*)

- Idempotency check on triggering_event_ref (the PaymentReceived event ID)
- Capture fresh customer_snapshot
- Build ONE line item for the direct payment:
 description = "Cycle pre-payment - {package_name}" (localized)
 package_ref = subscription's package
 service_category_code = subscription's service category
 total = paid_amount (tax-inclusive)
 Call PLM-CFG-02 Tax Configuration TaxService.computeTax(total, customer_location, service_category)
 → returns tax_breakdown decomposing the inclusive total
- Build invoice header:
 billing_mode = PREPAID
 customer_snapshot = fresh
 triggering_event_ref = PaymentReceived event ID
 original_invoice_id = pro forma ID (if exists) or NULL
 metadata.payment_received_event_id, payment_method
 grouping_config_ref = SINGLE-dimension group for TAX_INVOICE (or per operator config)
 period_start = target_cycle_start, period_end = target_cycle_end
 due_date = NULL, status = GENERATED, payment_status = PAID
- Allocate TAX_INVOICE sequence number atomically
- Write invoice + line items + outbox event
Result: tax invoice row in status GENERATED.

Step 4-6 – Signing, retries, downstream – see "Signing flow (all three flows)" below.

Signing flow (common to all three flows)

Step 4 – SigningScanner picks up the tax invoice

Every 1 minute per operator, scanner queries:

```

SELECT * FROM invoice
WHERE invoice_type_code = 'TAX_INVOICE'
AND status = 'GENERATED'
AND operator_code = :operator

```

For each row:

Step 4a – Transition status to PENDING_SIGNATURE (atomic compare-and-swap)

Step 4b – Look up signer implementation via invoice_type_operator_config.signing_service_implementation_ref

Step 4c – Call signer.sign(taxInvoice):

- Signer builds gateway-specific payload (proprietary format per authority)
- Submits over HTTPS with appropriate auth (mTLS / OAuth / signed JWT / etc.)
- Waits for response (timeout from operator config, default 30s)
- On success: parses response into TaxSignaturePayload
- On failure: throws TaxSigningException with category (TRANSIENT / VALIDATION)

Step 4d – On success:

- Update `invoice.metadata.tax_signature_data`, `signed_invoice_number`, `signed_at`
- Transition status to `SIGNED`
- Emit `TaxInvoiceSigned` event

Step 4e – On failure:

- Transition status to `SIGNING_FAILED`
- Insert `tax_invoice_signing_failure` row
- Set `next_retry_at` per failure category (transient → exponential backoff; validation → `NULL`)
- Emit `TaxInvoiceSigningFailed` event

Step 5 – `RetryScanner` picks up transient failures (R-TAX-01-F-2)
 Every 5 minutes per operator: re-submits `SIGNING_FAILED` with transient `failure_type` and `next_retry_at` ≤ `NOW`.
 After 8 transient failures: transitions to `GAVE_UP_AUTO` and emits `TaxInvoiceSigningGaveUp` event.

Step 6 – Downstream consumers
`TaxInvoiceSigned` → DD_NOT-01 Notification Service: renders PDF, sends to customer
`TaxInvoiceSigningGaveUp` → ops alerting
`TaxInvoiceSigningFailed` (VALIDATION) → admin queue: human action required

Failure paths:

Where it fails	Lands in	Recovery
TAX-01 consumer down	Kafka retains the event; consumed on restart	None needed (Kafka redelivery)
Tax invoice generation fails (DB, snapshot, BIL-01)	<code>generation_failure_queue</code> (same as GEN-01)	<code>GenerationFailureRetryScanner</code> (GEN-01 owns)
Gateway unreachable (network, DNS, 5xx)	Tax invoice in <code>SIGNING_FAILED</code> with <code>failure_type</code> <code>GATEWAY_UNREACHABLE</code>	Retry scanner
Gateway timeout	Same	Retry scanner
Gateway returns 4xx (validation)	Tax invoice in <code>SIGNING_FAILED</code> with <code>failure_type</code> <code>VALIDATION_FAILURE</code>	Admin queue — no auto retry
Gateway returns auth failure	Same with <code>failure_type</code> <code>AUTH_FAILURE</code>	Admin alert: certificate may have expired
8 transient retries exhausted	Tax invoice in <code>GAVE_UP_AUTO</code>	Admin manual review

The pluggable signer pattern

Every operator's tax authority has its own protocol. To support this without polluting the shared TAX-01 code with per-jurisdiction conditionals, TAX-01 uses an interface-based signer pattern.

The interface:

```
public interface TaxInvoiceSigner {

    /**
     * Returns the operator code this signer is registered for.
     * Used by the registry to select the right signer for a given operator.
     */
    String getOperatorCode();

    /**
     * Submit the tax invoice to the operator's tax authority gateway.
     * Returns a normalized payload containing the signed reference,
     * QR signature data, and gateway-specific metadata.
     * Throws TaxSigningException with category (TRANSIENT / VALIDATION) on failure.
     */
    TaxSignaturePayload sign Invoice taxInvoice) throws TaxSigningException;

    /**
     * Submit a cancellation request for an already-signed tax invoice.
     * Returns the gateway's cancellation reference on success.
     * Throws TaxSigningException on failure.
     */
    String cancel Invoice signedTaxInvoice, String cancellationReason) throws TaxSigningException;
}
```

Concrete implementations: one Java class per operator. The implementation knows:

- The gateway's URL and credentials (loaded from environment-specific config, NOT hardcoded)
- The wire protocol (REST, SOAP, fiscal device proxy, etc.)
- The payload schema (each authority has its own JSON or XML format)
- The authentication mechanism (mTLS client certs, OAuth client credentials, signed JWTs, etc.)
- Response parsing (extract signed reference number, QR data, timestamp)

- Error code mapping (each authority returns errors differently; the signer normalizes them to the `TaxSigningException` categories)

Signer registration: each implementation is a Spring `@Component`. At startup, the `TaxInvoiceSignerRegistry` scans for all `TaxInvoiceSigner` beans, indexes them by `getOperatorCode()`. At runtime, `TaxInvoiceSigningService` looks up the right signer based on the tax invoice's operator code.

What this means for adding a new operator: write a new signer class, annotate with `@Component`, deploy. No changes to the scanner, the retry framework, the data model, or other operators' signers. The pattern is the same as the `InvoiceGenerator` framework from BIL-02 Core.

Signer implementations at deployment time

Each operator that enables tax invoicing has one `TaxInvoiceSigner` implementation registered as a Spring `@Component`. The specific authority, protocol, payload format, and credentials are provided by the operator's tax team at deployment time. The TAX-01 framework supports any authority that fits the `TaxInvoiceSigner` contract; adding a new operator means writing one new class and deploying it — no changes to the scanner, retry framework, data model, or other operators' signers.

Operators whose deployment country does not require electronic tax invoice signing leave the tax invoice capability disabled (`invoice_type_operator_config.enabled = false` for `TAX_INVOICE`). For those operators, no signer is needed.

Additional data model (beyond Core)

The invoice table from Core is reused for tax invoice rows (with `invoice_type_code = TAX_INVOICE`). Signing-specific metadata lives in the `invoice.metadata` JSONB and (for detail audit) in this new table:

`tax_invoice_signing_failure`

One row per signing attempt that failed. Multiple rows per tax invoice are normal (each retry that fails creates a new row).

Column	Type	Notes
<code>id</code>	TEXT PK	ULID
<code>invoice_id</code>	TEXT NOT NULL	FK to the tax invoice in the <code>invoice</code> table
<code>attempt_number</code>	INTEGER NOT NULL	1, 2, 3, ... — sequential per invoice
<code>failed_at</code>	TIMESTAMPTZ NOT NULL	When the attempt failed
<code>failure_type</code>	TEXT NOT NULL	GATEWAY_UNREACHABLE / GATEWAY_TIMEOUT / GATEWAY_5XX / AUTH_FAILURE / VALIDATION_FAILURE / RATE_LIMITED / UNKNOWN (extensible)
<code>failure_category</code>	TEXT NOT NULL	TRANSIENT (auto-retry) / VALIDATION (admin-only)
<code>gateway_response_code</code>	TEXT NULL	Raw response code from the gateway (HTTP status, SOAP fault code, etc.)
<code>gateway_response_body</code>	TEXT NULL	Raw response body (truncated to 8KB for storage). Used for diagnosis.
<code>error_message</code>	TEXT NULL	Human-readable summary
<code>retry_scheduled_at</code>	TIMESTAMPTZ NULL	If transient: when the next retry will happen. NULL for validation failures.
<code>admin_resolution</code>	TEXT NULL	Notes from admin who reviewed (for validation failures)
<code>resolved_at</code>	TIMESTAMPTZ NULL	When admin resolved (if applicable)
<code>resolved_by</code>	TEXT NULL	Admin user ID

Indexes: (`invoice_id`, `attempt_number`) UNIQUE; (`failure_type`, `failed_at`) for analytics; (`failure_category`, `retry_scheduled_at`) partial WHERE `failure_category='TRANSIENT'` AND `retry_scheduled_at` IS NOT NULL — used by retry scanner. P10Y retention.

Sample data:

<code>id</code>	<code>invoice_id</code>	<code>attempt_number</code>	<code>failure_type</code>	<code>failure_category</code>	<code>gateway_response_code</code>	<code>error_message</code>	<code>retry_scheduled_at</code>
<code>tsf_01...A</code>	<code>inv_01HZQX8N...</code>	1	GATEWAY_TIMEOUT	TRANSIENT	NULL	Request timed out after 30s	2026-10-10 14:37:00
<code>tsf_01...B</code>	<code>inv_01HZQX8N...</code>	2	GATEWAY_TIMEOUT	TRANSIENT	NULL	Request timed out after 30s	2026-10-10 14:52:00

id	invoice_id	attempt_number	failure_type	failure_category	gateway_response_code	error_message	retry_scheduled_at
tsf_01...C	inv_01HZQX9P...	1	VALIDATION_FAILURE	VALIDATION	400	Customer tax identifier not recognized by authority	NULL

How to read: the first invoice failed twice with timeout (transient — retry scheduled). The second invoice has a validation failure (no retry scheduled; admin must fix the underlying issue in CRM and trigger manual re-signing).

API surface — what TAX-01 exposes

Mechanism	Used for	Examples
Kafka consumption (events in)	Trigger tax invoice generation from payment events	PaymentApplied (POSTPAID invoice payments), WalletToppedUp (PREPAID wallet topups), PaymentReceived (PREPAID direct-pay)
Scheduled scanners (no inbound API)	Signing scanner (every 1 min), retry scanner (every 5 min)	System-initiated, not external
Kafka emission (events out)	Notify downstream of tax invoice lifecycle	TaxInvoiceIssued, TaxInvoiceSigned, TaxInvoiceSigningFailed, TaxInvoiceSigningGaveUp, TaxInvoiceCancelled
REST (admin-facing)	Operational endpoints for ops/tax compliance staff	Manual generation, retry, cancel, dashboard queries
Outbound HTTPS	Tax authority gateway submissions	Per-signer protocol (mTLS, OAuth, etc.)

What TAX-01 does NOT expose as REST:

Capability	Where it lives instead
GET /api/tax-invoices/{id} (read tax invoice)	BIL-02-READ-01 Invoice Read API (tax invoices are just invoices with type=TAX_INVOICE; reading is unified there)
GET /api/tax-invoices/{id}/pdf	BIL-02-READ-01 Invoice Read API
POST /api/payments/... (record payment)	BIL-01 Charging Engine
POST /api/cycles/.../settle	BIL-05 Wallet and Topup
Tax computation rules / rates	PLM-CFG-02 Tax Configuration

Events (Kafka) — TAX-01

Emits:

Topic	When	Consumers
sophix.billing.taxinvoice.issued	Tax invoice row written (status GENERATED)	analytics
sophix.billing.taxinvoice.signed	Signing successful (status → SIGNED)	DD_NOT-01 Notification Service (render + send PDF), BIL-01 Charging Engine (audit), analytics
sophix.billing.taxinvoice.signingfailed	A signing attempt failed	ops alerting (if validation), retry scanner (internal — uses DB scan, not event)
sophix.billing.taxinvoice.signing.gaveup	After 8 transient retries → GAVE_UP_AUTO	ops alerting
sophix.billing.taxinvoice.cancelled	Tax invoice cancelled	BIL-01 Charging Engine (audit), DD_NOT-01 Notification Service (notify customer if appropriate), analytics

Consumes:

Topic	From	Action
sophix.billing.payment.applied	BIL-01 Charging Engine	Trigger tax invoice generation for POSTPAID invoice payments (full or partial)
sophix.billing.wallet.toppedup	BIL-05 Wallet and Topup	Trigger tax invoice generation for PREPAID wallet topups
sophix.billing.payment.received	BIL-01 Charging Engine	Trigger tax invoice generation for PREPAID direct-pay payments (no-wallet operator model)

REST endpoints (admin-facing — operational only)

GET /api/tax-invoices/dashboard?operator={code}

← ops dashboard summary

```

GET  /api/tax-invoices/{id}/signing-history
POST /api/tax-invoices/{id}/retry-signing
POST /api/tax-invoices/{id}/cancel
POST /api/tax-invoices/{id}/cancel/approve
POST /api/tax-invoices/{id}/resolve-no-action
POST /api/tax-invoices/manual

```

```

← all attempts + failures for one invoice
← manual retry
← cancel (signed needs dual approval)
← Tax Compliance Officer approval for signed cancellation
← mark failed invoice as offline-resolved
← admin manually generates a tax invoice for a payment

```

Cross-DD coordination

Touched	What TAX-01 owns	What the other DD owns
BIL-02 Invoicing (Core)	Implements TaxInvoiceGenerator per Core's framework; uses sequence service, line builder, writer, snapshot service, event publisher. Reuses invoice and invoice_line_item tables for tax invoice rows.	Owns framework + tables + lifecycle.
BIL-01 Charging Engine	Consumes PaymentApplied (POSTPAID invoice payments) and PaymentReceived (PREPAID direct-pay). Calls BIL-01 read APIs to load the parent invoice details for POSTPAID flow.	Owns payment recording + charge computation.
BIL-05 Wallet and Topup	Consumes WalletToppedUp for PREPAID wallet topups.	Owns wallet + topup mechanics.
BIL-02-GEN-01 Invoice Generation	Independent — no direct calls. GEN-01's bulk reversal explicitly excludes TAX_INVOICE (TAX-01 owns tax invoice cancellation through its own dual-approval flow).	Owns regular invoice generators.
PLM-CFG-01 Package Catalog	For PREPAID direct-pay: looks up the subscription's package → service category to tag the tax invoice line item.	Owns package and service-category mapping.
PLM-CFG-03 Wallet Configuration	For PREPAID wallet topup: resolves wallet_type_code → service category.	Owns wallet type catalog.
BIL-02-ADJ-01 Invoice Adjustments	None directly. (Credit notes against a tax invoice would require special handling — out of scope for V3 launch.)	Owns adjustment workflow.
BIL-02-READ-01 Invoice Read API	None directly. Tax invoice PDF retrieval flows through READ-01 via the unified invoice read API.	Owns PDF retrieval.
BIL-02-STA-01 Status Management & Recovery	None directly. Tax invoices don't have due dates and aren't subject to overdue/dunning logic; STA-01 explicitly excludes them.	Owns lifecycle for invoices with due dates.
SUB-LM-01 Subscription Lifecycle Management	Reads subscription state during generation (PENDING gate).	Owns subscription master.
PLM-CFG-02 Tax Configuration	For PREPAID topup and direct-pay: calls TaxService.computeTax(tax_inclusive_total, customer_location, service_category) to decompose the inclusive amount into base + tax components per the cascade. For POSTPAID: no PLM-CFG-02 call — tax is proportionally scaled from the parent invoice's already-computed tax breakdown (which GEN-01 obtained from PLM-CFG-02 at invoice generation time).	Owns tax rules (rates, cascades, exemptions).
CRM / Customer Service	Called via Core's CustomerSnapshotService for customer snapshot.	Owns live customer master.
DD_NOT-01 Notification Service	Emits TaxInvoiceSigned events with full payload; NOT-01 renders and sends.	Owns rendering + dispatch.
Tax authority gateways (external)	Calls per-signer protocol to the operator's tax authority API, where the operator has tax invoicing enabled.	External regulator-operated APIs.

Dependencies

Dependency	Owner
BIL-02 Invoicing (Core)	billing team (same wave)
BIL-01 Charging Engine	billing team
BIL-05 Wallet and Topup	billing team
SUB-LM-01 Subscription Lifecycle Management	subscription team
PLM-CFG-02 Tax Configuration	catalog team
CRM / Customer Service module	CRM team

Dependency	Owner
DD_NOT-01 Notification Service	notification team
FOUNDATION_DB.md, FOUNDATION_KAFKA.md, FOUNDATION_AUTH.md	infrastructure team
Per-operator tax authority API specifications	each operator's tax team
Per-operator certificates / OAuth credentials (provided by tax authorities)	each operator's tax team (handed to infrastructure for vault storage)

Operator-configurable capabilities (specific to TAX-01)

Capability	Where configured	Notes
Tax invoice enablement	<code>invoice_type_operator_config.enabled</code> for TAX_INVOICE row	Operator chooses to enable; if false, TAX-01 acknowledges trigger events but generates nothing
Signing service implementation	<code>invoice_type_operator_config.signing_service_implementation_ref</code>	The Spring bean name of the signer class for this operator
Signing timeout	Per-signer config (gateway-specific)	Default 30 seconds; operator can override per gateway behavior
Retry backoff schedule	Per-signer config	Default 5min/15min/30min/1h/2h/4h/8h/24h; operator can tune
Max transient retries	Per-signer config	Default 8; operator can adjust based on gateway reliability
Tax invoice PDF template	Per-operator template in DD_NOT-01 Notification Service	Tax-authority-mandated layouts; each operator's template includes the QR code and signed reference
Operator country-specific tax fields	Captured in customer snapshot at issuance	<code>tax_identifier.type</code> per the operator's jurisdiction; operator-specific business registration number; other authority-required fields
Tax reporting segregation per package / service	Line items carry <code>service_category_code</code> , <code>package_ref</code> , <code>wallet_type_code</code> ; tax reporting queries aggregate by these dimensions	Operator finance team can produce per-service-type and per-package tax reports for reconciliation with authority filings
Per-operator regulatory pre-go-live declaration	Coordinated with PLM-CFG-01 Package Catalog change-management workflow	Some operators must declare discounts, promotions, price changes, and product code changes to their regulatory authority before activation. PLM-CFG-01 owns the workflow that gates such changes behind regulator-declaration confirmation; TAX-01 receives the activated configuration and applies it at tax invoice generation. The specific declaration requirements are per-operator.

B — Current TZ

Tax invoice generation does not exist in TZ Sophix. No prior implementation to describe or compare against.

C — Gap & bridge

TAX-01 is entirely new functionality. There is no TZ baseline to bridge from. All capabilities described in section A are introduced fresh by V3.